

Documentation d'Installation

Nextcloud Whiteboard sur Raspberry Pi

L'installation se déroule sur un Raspberry Pi tournant sous Debian. La documentation officielle de Docker pour Debian est utilisée comme référence :

<https://docs.docker.com/engine/install/debian/#install-using-the-repository>

<https://help.nextcloud.com/t/how-do-i-setup-the-websocket-server-for-whiteboard-real-time-collaboration/229171>

1.1 Mise à jour du système

Les deux premières commandes exécutées permettent de synchroniser et mettre à jour les paquets du système

Commandes exécutées :

```
sudo apt-get update  
sudo apt-get upgrade
```

```
amspt05admin@RaspberryPi-AMSP05:~ $ sudo apt-get update  
Réception de : 1 http://deb.debian.org/debian trixie InRelease [140 kB]  
Réception de : 2 http://deb.debian.org/debian-security trixie-security InRelease  
[43,4 kB]  
Réception de : 3 http://archive.raspberrypi.com/debian trixie InRelease [54,9 kB]  
Réception de : 4 http://deb.debian.org/debian trixie-updates InRelease [47,3 kB]  
Réception de : 5 http://deb.debian.org/debian trixie/main armhf Packages [9238  
kB]  
Réception de : 6 http://archive.raspberrypi.com/debian trixie/main armhf Package  
s [443 kB]  
Réception de : 7 http://archive.raspberrypi.com/debian trixie/main arm64 Package  
s [449 kB]  
Réception de : 8 http://deb.debian.org/debian trixie/main arm64 Packages [9608  
kB]  
Réception de : 9 http://deb.debian.org/debian trixie/main Translation-en [6485  
kB]  
Réception de : 10 http://deb.debian.org/debian-security trixie-security/main arm  
64 Packages [177 kB]  
Réception de : 11 http://deb.debian.org/debian-security trixie-security/main arm  
hf Packages [167 kB]  
Réception de : 12 http://deb.debian.org/debian-security trixie-security/main Tra  
nslation-en [111 kB]  
27,0 Mo réceptionnés en 3s (8848 ko/s)  
Lecture des listes de paquets... Fait  
N: Le dépôt « http://deb.debian.org/debian trixie InRelease » a modifié sa valeu  
r « Version » de « 13.4 » à « 13.5 »  
amspt05admin@RaspberryPi-AMSP05:~ $ sudo apt-get upgrade  
Lecture des listes de paquets... Fait
```

1.2 Installation des prérequis pour Docker

Avant d'ajouter le dépôt Docker, il faut installer les outils de gestion des certificats et de téléchargement HTTPS.

commande :

```
sudo apt-get install ca-certificates curl
```

1.3 Ajout de la clé GPG Docker

Pour sécuriser les téléchargements depuis le dépôt Docker, une clé de signature GPG est ajoutée au trousseau de confiance du système.

```
sudo install -m 0755 -d /etc/apt/keyrings
```

(Crée le dossier de stockage des clés APT avec les permissions correctes (rwxr-xr-x))

```
sudo curl -fsSL https://download.docker.com/linux/raspbian/gpg -o /etc/apt/keyrings/docker.asc
```

(Télécharge la clé GPG officielle de Docker pour Raspberry Pi / Debian ARM.)

```
amspt05admin@RaspberryPi-AMSP05:~$ sudo apt-get install ca-certificates curl
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
ca-certificates est déjà la version la plus récente (20250419).
curl est déjà la version la plus récente (8.14.1-2+deb13u3).
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  pigpio-tools
Veuillez utiliser « sudo apt autoremove » pour le supprimer.
0 mis à jour, 0 nouvellement installés, 0 à enlever et 5 non mis à jour.
amspt05admin@RaspberryPi-AMSP05:~$ sudo install -m 0755 -d /etc/apt/keyrings
amspt05admin@RaspberryPi-AMSP05:~$ sudo curl -fsSL https://download.docker.com/linux/raspbian/gpg -o /etc/apt/keyrings/docker.asc
```

Le dépôt Docker officiel est ensuite ajouté aux sources APT, puis Docker et ses composants sont installés.

commande :

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

(Rend la clé lisible par tous les utilisateurs du système)

1.4 Ajout du dépôt Docker et installation

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]  
https://download.docker.com/linux/raspbian \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
amspt05admin@Raspberrypi-AMSPT05:~$ sudo curl -fsSL https://download.docker.com/linux/raspbian/gpg -o /etc/apt/keyrings/docker.asc  
amspt05admin@Raspberrypi-AMSPT05:~$ ^[[200~sudo chmod a+r /etc/apt/keyrings/docker.asc  
-bash: $'\E[200~sudo': commande introuvable  
amspt05admin@Raspberrypi-AMSPT05:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc  
amspt05admin@Raspberrypi-AMSPT05:~$ echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/raspbian \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Installer Docker Engine complet :
commande :

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin
```

```
amspt05admin@Raspberrypi-AMSPT05:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin  
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :  
  pigpio-tools  
Veuillez utiliser « sudo apt autoremove » pour le supprimer.  
  
Installation de :  
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-compose-plugin  
  
Installation de dépendances :  
  docker-ce-rootless-extras git git-man liberror-perl pigz  
  
Paquets suggérés :  
  cgroupfs-mount | cgroup-lite docker-model-plugin git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn  
  
Sommaire :  
  Mise à niveau de : 0. Installation de : 10Supprimé : 0. Non mis à jour : 5  
Taille du téléchargement : 96,9 MB  
  Espace nécessaire : 409 MB / 71,5 GB disponible  
  
Continuer ? [O/n] o  
Réception de : 1 http://deb.debian.org/debian trixie/main arm64 pigz arm64 2.8-1+b1 [56,3 kB]  
Réception de : 2 http://deb.debian.org/debian trixie/main arm64 liberror-perl all 0.17030-1 [26,9 kB]  
Réception de : 3 http://deb.debian.org/debian trixie/main arm64 git-man all 1:2.47.3-0+deb13u1 [20205 kB]  
Réception de : 4 https://download.docker.com/linux/debian trixie/stable arm64 containerd.io arm64 2.2.4-1~debian.13~trixie [20,2 MB]  
Réception de : 5 http://deb.debian.org/debian trixie/main arm64 git arm64 1:2.47.3-0+deb13u1 [80666 kB]
```

1.5 Vérification de l'installation Docker

Après l'installation, le statut du service Docker est vérifié et un test de fonctionnement est effectué.

```
sudo systemctl status docker
```

```
amspt05admin@RaspberryPi-AMSPT05:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-05-27 14:20:55 CEST; 1min 58s ago
  Invocation: 69a64cbe7677497e84ce7bb7a52a086c
  TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 731392 (dockerd)
      Tasks: 11
         CPU: 218ms
    CGroup: /system.slice/docker.service
            └─731392 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.461772983+02:00" level=info msg="Deleting iptables IPv6 rules" error="exit status 1" output="Error: No such file or directory"
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.645188125+02:00" level=info msg="Loading containers: done."
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.652596326+02:00" level=warning msg="WARNING: No memory limit support"
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.652622290+02:00" level=warning msg="WARNING: No swap limit support"
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.652645956+02:00" level=info msg="Docker daemon" commit=568f755 containerd-snapshotter=true storage=overlayfs2
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.652748879+02:00" level=info msg="Initializing buildkit"
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.668830814+02:00" level=info msg="Completed buildkit initialization"
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.676977762+02:00" level=info msg="Daemon has completed initialization"
mai 27 14:20:55 RaspberryPi-AMSPT05 dockerd[731392]: time="2026-05-27T14:20:55.677059874+02:00" level=info msg="API listen on /run/docker.sock"
mai 27 14:20:55 RaspberryPi-AMSPT05 systemd[1]: Started docker.service - Docker Application Container Engine.
```

```
sudo docker run hello-world
```

```
amspt05admin@RaspberryPi-AMSPT05:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
58dee6a49ef1: Pull complete
c3bdf82c34d1: Download complete
Digest: sha256:0e760fd9bc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(arm64v8)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

1.6 Création du dossier Whiteboard

Un dossier dédié est créé pour stocker la configuration de l'application Whiteboard.

```
sudo mkdir /opt/whiteboard
```

```
sudo: mkdir/opt/whiteboard : commande introuvable  
amspt05admin@RaspberryPi-AMSPT05:~$ sudo mkdir /opt/whiteboard
```

Le fichier compose.yaml définit la configuration du container Docker pour Nextcloud Whiteboard :

création :

```
sudo nano /opt/whiteboard/compose.yaml
```

```
amspt05admin@RaspberryPi-AMSPT05:/opt/whiteboard$ sudo nano /opt/whiteboard/compose.yaml  
amspt05admin@RaspberryPi-AMSPT05:/opt/whiteboard$
```

et nous ajoutons ça dans le fichier :

```
services:  
  nextcloud-whiteboard-server:  
    image: ghcr.io/nextcloud-releases/whiteboard:stable  
    ports:  
      - "3002:3002"  
    environment:  
      NEXTCLOUD_URL: https://nextcloud.assolsf05.fr  
      JWT_SECRET_KEY: ${WBSECRET}
```

```
GNU nano 8.4 compose.yaml  
services:  
  nextcloud-whiteboard-server:  
    image: ghcr.io/nextcloud-releases/whiteboard:stable  
    ports:  
      - "3002:3002"  
    environment:  
      NEXTCLOUD_URL: https://nextcloud.assolsf05.fr  
      JWT_SECRET_KEY: ${WBSECRET}
```

1.7 Edition du fichier .env

```
sudo nano .env
```

```
amspt05admin@RaspberryPi-AMSPT05:/opt/whiteboard $ sudo nano .env
```

```
WBSECRET= secret a définir
```

```
GNU nano 8.4 /opt/whiteboard/.env
WBSECRET=
```

1.8 Démarrage du container Docker Compose

Le container Whiteboard est démarre en mode détaché (arriere-plan) via Docker Compose.

```
sudo docker compose up -d
```

```
amspt05admin@RaspberryPi-AMSPT05:/opt/whiteboard $ sudo docker compose up -d
[+] up 10/10
  ✓ Image ghcr.io/nextcloud-releases/whiteboard:release Pulled
  ✓ Network whiteboard_default Created
  ✓ Container whiteboard Started
amspt05admin@RaspberryPi-AMSPT05:/opt/whiteboard $
```

1.9 Configuration du Virtual Host Apache

Pour exposer le service Whiteboard via un nom de domaine, un Virtual Host Apache est configuré. Le fichier édité est :

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

et il faut ajouter ça dans le fichier:

```
ProxyPass /whiteboard http://127.0.0.1:3002 upgrade=websocket
ProxyPassReverse /whiteboard http://127.0.0.1:3002 upgrade=websocket
```

```
GNU nano 8.4 /etc/apache2/sites-available/000-default.conf
```

```
<VirtualHost *:80>
# The ServerName directive sets the request scheme, hostname and port that
# the server uses to identify itself. This is used when creating
# redirection URLs. In the context of virtual hosts, the ServerName
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerName nextcloud.assolsf05.fr
ServerAdmin contact@assolsf05.fr
DocumentRoot /var/www/nextcloud

# Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the loglevel for particular
# modules, e.g.
#LogLevel info ssl:warn

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

# For most configuration files from conf-available/, which are
# enabled or disabled at a global level, it is possible to
# include a line for only one particular virtual host. For example the
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf
#RewriteEngine on
#RewriteCond %{SERVER_NAME} =assolsf05.fr [OR]
#RewriteCond %{SERVER_NAME} =nextcloud.assolsf05.fr
#RewriteRule ^ https://%{SERVER_NAME}%{REQUEST_URI} [END,NE,R=permanent]

ProxyPass /whiteboard http://127.0.0.1:3002 upgrade=websocket
ProxyPassReverse /whiteboard http://127.0.0.1:3002 upgrade=websocket
</VirtualHost>
```